Double Descent 2: Feature learning and Frozen kernel Model-Wise Double Descents

Viktor Veselý *v.vesely@student.rug.nl*
University of Groningen

Abstract

1 Introduction

Model-wise double descent challenges the expectation that increasing capacity beyond interpolation necessarily harms generalization. As models grow, validation error can decrease, peak near interpolation, and decrease again despite fitting the training noise. Two streams of literature seek to explain this curve: frozen-kernel analyses of the instability of interpolation, and empirical studies of deep networks that learn their representations. Their common target suggests a common mechanism; we show that the same curve shape can arise from two distinct mechanisms.

Frozen-kernel explanations locate the peak in the amplification of noise by weakly constrained feature directions. Near interpolation, fitting residual errors along these directions requires large coefficients that can transfer poorly to unseen data; greater capacity can reduce this sensitivity (Bach, 2023; Schaeffer et al., 2023). A finite-width extension allows features to change but assumes that the smallest nonzero eigenvalue of their gradient covariance remains comparable to its initial value, retaining a spectral link to initialization (Singh et al., 2022). More recently, a theory accommodating evolving kernels also recovers double descent through a frozen-kernel specialization (Litman & Guo, 2026). These explanations describe how noise is fitted within an available representation.

Empirical deep-learning studies instead observe double descent as the representation itself changes with training and model width. Analyses of learned feature neighborhoods associate the second descent with improved separation of corrupted labels from class structure (Gu et al., 2024). This raises a complementary question about the peak: can learning the representation itself create the deterioration in generalization? Resolving this question distinguishes a peak caused by feature formation from the interpolation instability described by frozen-kernel theory.

Learning features that facilitate noise fitting provides a candidate mechanism for this empirical peak. The closest work characterizes a transition from signal learning to memorization of sample-specific noise in a two-layer convolutional network (Han et al., 2025). Its focus is the evolution of signal and noise learning during optimization. Our focus is how such feature changes generate a nonmonotonic generalization curve across model widths, which samples induce them, and whether suppressing their feature-learning contribution reduces the peak while preserving their fit. Related observations of clean- and noisy-label learning in epoch-wise double descent further motivate distinguishing the progress of noise fitting from its consequences across model sizes (KUBO & IIDA, 2025).

We identify two distinct mechanisms of model-wise double descent within the same task and architecture. On MNIST and CIFAR-10 with label corruption, *feature-learning double descent* (FLDD) arises earlier as learned features accelerate noise fitting, while *frozen-kernel double descent* (FKDD) emerges at longer horizons through slowly learned, weakly constrained directions. Matched nonlinear and frozen-feature models distinguish the mechanisms. Freezing trained features and evaluating their finite-time kernel dynamics reveals a later FKDD peak alongside the existing FLDD peak (Figure 1). This connects the two literature streams through distinct mechanisms that can coexist, rather than treating their explanations as interchangeable.

We test the feature-learning mechanism by intervening on the samples that drive noise-fitting feature changes. We attribute these changes to individual examples, which may be clean or noisy, and selectively disable their contribution to feature learning. The selected examples remain trainable through a readout, preserving the ability to fit them. The resulting reduction in the peak links FLDD to the formation of noise-fitting features and provides a targeted experimental test of the proposed mechanism.

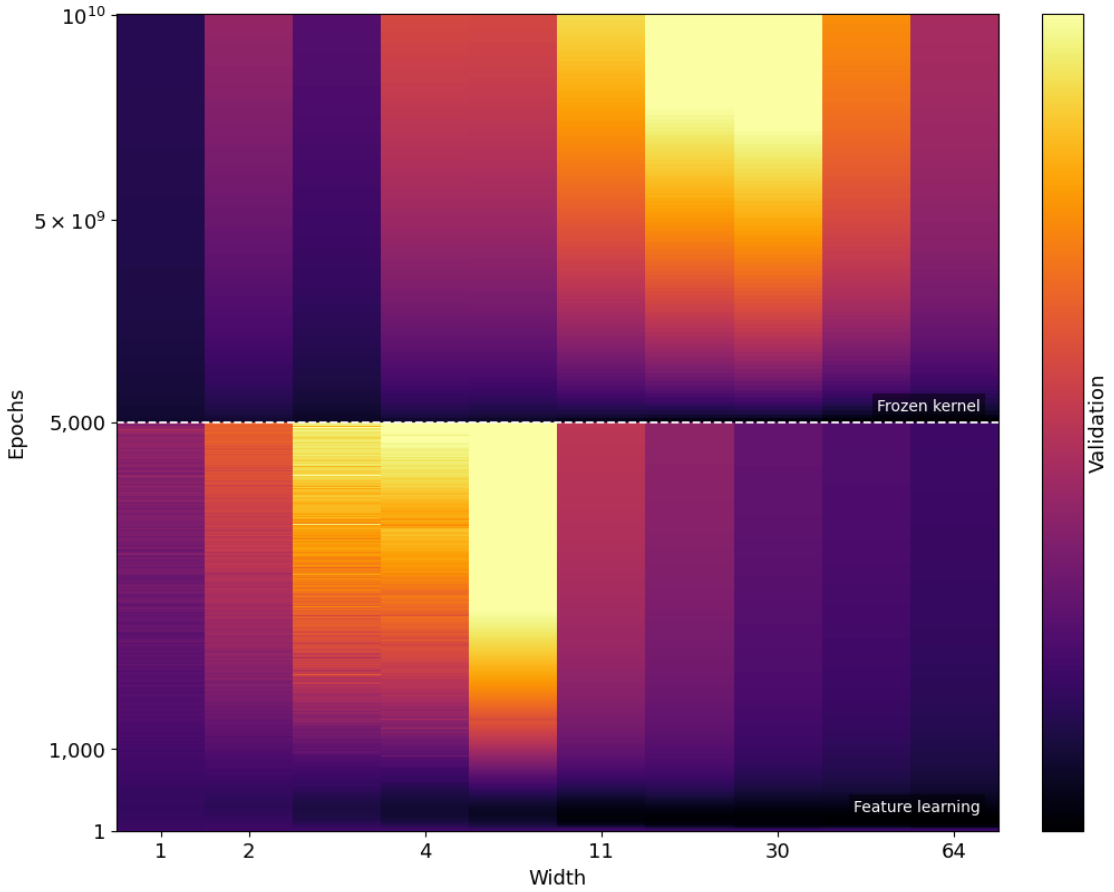


Figure 1: Two mechanisms of model-wise double descent for the same task and architecture. The lower part shows the earlier feature-learning double descent; the upper part shows the later frozen-kernel double descent revealed by fixed-feature continuation. The peaks occur at different widths and training horizons, separating the effect of feature formation from the slow fitting of kernel modes.

1.1 Contributions

Our contributions are twofold:

- **(C1) Identifying two distinct mechanisms of model-wise double descent.** We distinguish FLDD from FKDD through matched feature-learning ablations and demonstrate their coexistence through frozen-kernel dynamics evaluated at trained checkpoints.

- **(C2) Identifying and intervening on the feature-learning mechanism.** We derive sample-level scores for feature changes that facilitate noise fitting and reduce the FLDD peak by selectively disabling these changes while preserving the ability to fit the selected samples.

2 Background

We consider a supervised learning setting with a training dataset $\mathbb{S} = \{\boldsymbol{x}_s, \boldsymbol{y}_s\}_{l=1}^{S}$ of S i.i.d. samples, where $\boldsymbol{x} \in \mathbb{R}^{d_x}$ denotes the input and $\boldsymbol{y} \in \mathbb{R}^{d_y}$ the corresponding real valued output for regression and $\boldsymbol{y} \in \{0,1\}^{d_y}$ the one-hot vector over d_y classes for classification. Analogously, $\mathbb{Q} = \{\boldsymbol{x}_q, \boldsymbol{y}_q\}_{q=1}^{Q}$ defines the testing dataset with Q samples.

We define $\boldsymbol{\theta}_{n,p} \in \mathbb{R}^p$ to be p parameters of a neural network $f_{\boldsymbol{\theta}}$ trained for n epochs. We train the parameters by minimizing the training loss $\mathcal{L}_{\mathbb{S}}$ which is instantiated to MSE for regression tasks and CE for classification tasks. Test loss $\mathcal{L}_{\mathbb{Q}}$ is used to measure the final performance.

We train our models with the plain stochastic gradient descent for random batches $\mathbb{B} \subset \mathbb{S}$ and a constant learning rate η, yielding the update rule $\theta_{t+1} = \theta t + \eta \nabla_{\theta_t} \mathcal{L}_{\mathbb{B}}(\theta_t)$.

To induce double descent in classification tasks, we introduce label noise at a rate $\xi \in [0,1]$. Before training, we select a fraction ξ of the training samples uniformly at random and replace each selected label with a label drawn uniformly from the remaining $d_y - 1$ classes for classification or drawn uniformly from other samples otputs for regression. Both the selected samples and their corrupted labels remain fixed throughout training, while the test labels remain unchanged.

2.1 Taylor approximation of the SGD learning dynamics

Definitions We examine the change in loss induced by an SGD update on a training batch $\mathbb{B} \subset \mathbb{S}$. To distinguish the contribution of the current tangent features from that of their evolution, we express the update in terms of output gradients and parameter Jacobians. Unless stated otherwise, quantities below are evaluated at the current parameters $\boldsymbol{\theta}_t$, with this dependence suppressed for readability.

$$
\begin{aligned}
\boldsymbol{b} \coloneqq (f_{\boldsymbol{\theta}}(\boldsymbol{x}_1) : \ldots : f_{\boldsymbol{\theta}}(\boldsymbol{x}_{|\mathbb{B}|})), \quad \boldsymbol{x}_i \in \mathbb{B} && \text{Batch outputs' concatenation;} \\
\boldsymbol{g}_{\mathbb{B}} \coloneqq \nabla_{\boldsymbol{b}} \mathcal{L}_{\mathbb{B}} && \text{Output training gradient;} \\
\boldsymbol{J}_{\mathbb{B}} \coloneqq \nabla_{\theta} \boldsymbol{b} && \text{Jacobian of batch otuputs;} \\
\boldsymbol{p}_{\mathbb{B}} \coloneqq \boldsymbol{J}_{\mathbb{B}}^{\top} \boldsymbol{g}_{\mathbb{B}} && \text{Parameter training gradient for the batch.} \\
K_{\mathbb{X}\mathbb{Y}} \coloneqq \boldsymbol{J}_{\mathbb{X}} \boldsymbol{J}_{\mathbb{Y}}^{\top} && \text{Cross-kernel matrix between set } \mathbb{X} \text{ and set } \mathbb{Y}
\end{aligned}
$$

Local loss dynamics For the update $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \boldsymbol{p}_{\mathbb{B}}$, we define the batch-loss change as $\Delta \mathcal{L}_{\mathbb{B}} \coloneqq \mathcal{L}_{\mathbb{B}}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}_{\mathbb{B}}(\boldsymbol{\theta}_t)$. We approximate this change by a second-order Taylor expansion along the straight parameter-space path of the update. The update direction is held fixed along this path; the dots denote directional derivatives evaluated at $\boldsymbol{\theta}_t$. The resulting approximation separates the first-order change from two contributions to the second-order change:

$$
\Delta \mathcal{L}_{\mathbb{B}} \approx \Delta t \dot{\mathcal{L}}_{\mathbb{B}} + \frac{(\Delta t)^2}{2} \ddot{\mathcal{L}}_{\mathbb{B}} \tag{1}
$$

$$
\dot{\mathcal{L}}_{\mathbb{B}} = -\boldsymbol{g}_{\mathbb{B}}^{\top} \boldsymbol{J}_{\mathbb{B}} \boldsymbol{J}_{\mathbb{B}}^{\top} \boldsymbol{g}_{\mathbb{B}} \tag{2}
$$

$$
\ddot{\mathcal{L}}_{\mathbb{B}} = \underbrace{\boldsymbol{p}_{\mathbb{B}}^{\top} \boldsymbol{J}_{\mathbb{B}}^{\top} (\nabla_f^2 \mathcal{L}_{\mathbb{B}}) \boldsymbol{J}_{\mathbb{B}} \boldsymbol{p}_{\mathbb{B}}}_{\text{Loss curvature}} + \underbrace{\boldsymbol{p}_{\mathbb{B}}^{\top} \left(\sum_{i=1}^{|\mathbb{B}| d_y} (\boldsymbol{g}_{\mathbb{B}})_i \nabla_{\boldsymbol{\theta}}^2 b_i \right) \boldsymbol{p}_{\mathbb{B}}}_{\text{Feature-learning curvature}} \tag{3}
$$

Here the step length equals the learning rate, $\Delta t = \eta$. The first-order term describes descent through the current tangent features. At second order, the loss-curvature term accounts for curvature of the loss with respect to the model outputs, whereas the feature-learning curvature term depends on the second derivatives of those outputs with respect to the parameters. Second order is thus the lowest order at which output curvature appears explicitly in this local expansion. Appendix A.2 provides the derivation, and Appendix A.4 provides the implemented matrix-free versions of the presented terms.

Equation 1 is a local approximation. Its accuracy over a finite SGD step depends on how the directional derivatives vary along the update path; a second-order expansion at the starting point need not adequately capture the full loss change. This motivates integration along the update path, as discussed in Appendix A.5.

For a frozen kernel models the feature-learning curvature term vanishes, explicitly identifying the difference between feature learning and frozen kernel models. The feature learning term misses kernel changes that are locally independent to the loss changes. To complement this directional quantity, we measure changes in the training kernel over the course of optimization. We track the training kernel $K_t = J_{\mathbb{S}}(\boldsymbol{\theta}_t)J_{\mathbb{S}}(\boldsymbol{\theta}_t)^\top$ using two relative Frobenius-norm metrics: its drift from initialization and its change since the previous measurement t^-,

$$D_t = \frac{\|K_t - K_0\|_F}{\|K_0\|_F}, \qquad S_t = \frac{\|K_t - K_{t^-}\|_F}{\|K_0\|_F}. \tag{4}$$

The first metric measures cumulative drift from initialization, while the second measures change between successive measurements.

Transfer to validation loss The effect of a training update on validation loss is determined, to first order, by the alignment between the batch and validation parameter gradients. Let $\boldsymbol{g}_{\mathbb{Q}}$ denote the gradient of $\mathcal{L}_{\mathbb{Q}}$ with respect to the concatenated validation outputs, and let $\boldsymbol{J}_{\mathbb{Q}}$ denote their parameter Jacobian. Along the same SGD update path, the chain rule gives

$$\dot{\mathcal{L}}_{\mathbb{Q}} = -\big(\boldsymbol{J}_{\mathbb{Q}}^\top \boldsymbol{g}_{\mathbb{Q}}\big)^\top \boldsymbol{p}_{\mathbb{B}} = -\boldsymbol{g}_{\mathbb{Q}}^\top K_{\mathbb{Q}\mathbb{B}} \boldsymbol{g}_{\mathbb{B}}. \tag{5}$$

To obtain interpretable scalar metrics for each training sample's contribution to validation learning, we separate its output-gradient magnitude from its transfer to validation. Let $\boldsymbol{g}_{\mathbb{B}}^{(i)}$ be its block of the batch output gradient and $(\boldsymbol{J}_{\mathbb{B}}^\top)^{(i)}$ the corresponding block of the transposed Jacobian. Writing $\hat{\boldsymbol{g}}_{\mathbb{B}}^{(i)} = \boldsymbol{g}_{\mathbb{B}}^{(i)}/\|\boldsymbol{g}_{\mathbb{B}}^{(i)}\|_2$, with $\hat{\boldsymbol{g}}_{\mathbb{B}}^{(i)} = 0$ when the gradient vanishes, we define the *Train push* P_i and *Transfer* T_i through the decomposition

$$\dot{\mathcal{L}}_{\mathbb{Q}} = -\sum_{i=1}^{|\mathbb{B}|} \underbrace{\|\boldsymbol{g}_{\mathbb{B}}^{(i)}\|_2}_{P_i \text{ (Train push)}} \underbrace{\boldsymbol{g}_{\mathbb{Q}}^\top \boldsymbol{J}_{\mathbb{Q}} (\boldsymbol{J}_{\mathbb{B}}^\top)^{(i)} \hat{\boldsymbol{g}}_{\mathbb{B}}^{(i)}}_{T_i \text{ (Transfer)}} = -\sum_{i=1}^{|\mathbb{B}|} P_i T_i. \tag{6}$$

Train push P_i measures the magnitude of the sample's output gradient, while Transfer T_i measures the alignment of the corresponding parameter direction with the validation gradient. Positive T_i therefore contributes to a first-order decrease in validation loss, and negative T_i contributes to an increase. This decomposition separates how strongly a sample drives the update from how beneficial that direction is for validation performance.

2.2 Kernel regression

We use finite-time kernel regression to evaluate how validation performance evolves when the current features remain fixed. At a chosen checkpoint, we freeze their parameter Jacobian, obtaining an affine model of subsequent parameter changes. We evaluate the evolution of this model under full-batch gradient descent on a squared-error objective. This allows us to explore long training horizons without explicitly simulating every update, including the slow learning of kernel modes with small eigenvalues. Computational details are given in Appendix A.4.

We apply this continuation both at initialization and after training, across model widths and random seeds. Freezing at initialization isolates frozen-kernel double descent (FKDD), while freezing a trained network that already exhibits feature-learning double descent (FLDD) allows us to examine whether a later FKDD develops alongside it. These continuations separate the effects of continued fitting with fixed features from those of further feature learning.

For regression, we linearize the model outputs and use mean squared error. For classification, we linearize the softmax probabilities and fit the labels with Brier loss, reporting validation error averaged over output coordinates. The classification continuation therefore describes squared-error learning in the affine probability model, rather than continued cross-entropy training of the original network.

2.3 Frozen Kernel Double Descent mechanism

The mechanisms underlying frozen-kernel double descent have been studied extensively. Here, we follow the three-factor account of Schaeffer et al. (2023), applied to the frozen Jacobian as the feature matrix. The mechanism combines small nonzero eigenvalues of the empirical tangent kernel, residual error from noise or model misspecification aligned with the corresponding training modes, and validation features that amplify the resulting estimation error. Fitting this residual along weakly constrained directions requires large parameter coefficients, which can produce large prediction errors on validation samples. We refer to double descent arising through this mechanism as frozen-kernel double descent (FKDD).

This account describes FKDD through spectral modes, whereas our proposed feature-learning double descent (FLDD) mechanism focuses on how individual training samples drive changes in the features. Ridge regularization mitigates the FKDD mechanism by suppressing the amplification associated with small eigenvalues.

3 Methodology

3.1 Specification of the double descent curve

Model-wise double descent traces a reversal in the benefit of increasing model capacity: validation loss initially falls, rises to a peak, and then falls again as the model grows wider. To study how learning changes across this curve, we summarize its geometry at a fixed training horizon T by four ordered widths,

$$\mathcal{D}_T \coloneqq (w_{\mathrm{s}}, w_-, w_\star, w_+), \qquad w_{\mathrm{s}} < w_- < w_\star < w_+. \tag{7}$$

Here w_{s} is the smallest width considered, w_- marks the validation-loss minimum before the peak, $w_\star$ the peak itself, and w_+ the minimum reached beyond it within the explored range. Together, these landmarks place the deterioration in generalization between two reference models with lower validation loss, while retaining the starting point of the width sweep.

This four-point description gives us a common language for comparing learning dynamics across the curve and asking how an intervention changes its shape. We identify the landmarks from the final validation losses, requiring a clearly distinguished peak with lower loss on either side. The numerical identification procedure is given in Appendix A.1.

3.2 Identification of two Double Descent mechanisms

Showing the distinction We can identify the two distinct double double descents by starting from a common point: with the same architecture, task, data ordering, initial features, weight initializations, learning rate, epoch budget T etc. Then we continue with an feature learning ablation, we train two copies of the same starting point one with feature feature learning where the parameters fully free to move and the second copy is with frozen kernel, where the initial features stay the same. The cleanest way of freezing the kernel for a function $f_{\boldsymbol{\theta}_t}$ is to introduce a second parameter vector $\boldsymbol{w} \in \mathbb{R}^{|\boldsymbol{\theta}|}$ which learns the tangent features:

$$\bar{f}_{\boldsymbol{\theta}_t}(\boldsymbol{x}; \boldsymbol{w}) = f_{\boldsymbol{\theta}_t}(\boldsymbol{x}) + (\nabla_{\boldsymbol{\theta}_t} f_{\boldsymbol{\theta}_t})\boldsymbol{w} \tag{8}$$

where $\boldsymbol{\theta}_t$ will be frozen and $\boldsymbol{w}$ will be updated freely. Substituting $\boldsymbol{\theta}_0$ and learning showcases that the double descent dissapears for the frozen kernel regime (show a figure here).

To further prove the distinction of the two DD mechanims we can apply their respective interventions to each other and show that the DD peak does not dissapear.

(this section maybe needs to be moved somewhere I am just putting it here as a notes) Applying the FKDD intervention (regularization of the kernel) to the FLDD: - so there is no one to one correspondence of the ridge regression to the SGD training. - We have tried sofar the classical L2 penalty of the weights and sweeping the regularization strength we have found that - With strong regularization the FLDD peak dissapears, however, not because the intervention would work but because the strong L2 penalty forced the training into a lazy mode (almost frozen) this is also confirmed in Repasky et al. (2023) and Lewkowycz & Gur-Ari (2020), and

we empirically show the kernel movement to be substantially smaller for those cases (reference a figure) - With weak regulariztion the feature learning regime stays and the FLDD peak remains (reference a figure)

Applying the FLDD intervention to the frozen kernel regime would do nothing by definiton since we only disable the feature learning which is already disabled there.

Showing the coexistence The coexistence of the two DDs for the same full feature learning run is this: - for the tasks and architectures presented here it is computationally feasible to empirically observe the FKDD in the actual feature learning run since as seen from the figure the occurance of FKDD happens in very distant epochs (around $5 \cdot 10^9$) - thus we based the coexistence of the two DDs based on the assumption that the movement of the kernel (feature learning) slows down as the training progresses (which is not theoratically guaranteed but it is a common empirical behavior (Atanasov et al., 2021)) - Using this assumption we can freeze the actual learned model $\boldsymbol{\theta}_T$ by again using Equation 8 and using kernel regression to display the validation error given the frozen kernel assumption

3.3 Feature-Learning Double Descent Mechanism

We hypothesize that feature learning can give rise to double descent by creating representations that make label noise easier to fit. The proposed mechanism involves three conditions: (i) updates induced by some training samples create features that facilitate noise fitting; (ii) the network has sufficient capacity to fit the noise and does so rapidly once these features emerge; and (iii) fitting the noisy samples transfers poorly to validation data. We refer to double descent associated with this mechanism as *feature-learning double descent* (FLDD).

Feature changes that facilitate noise fitting. To formalize condition (i), let $\mathbb{L} \subseteq \mathbb{S}$ denote the noisy samples in the training dataset. Along the straight-line path of an SGD update, $\dot{\boldsymbol{\theta}} = -\boldsymbol{p}_{\mathbb{B}}$, the quantity $\boldsymbol{g}_{\mathbb{L}}^\top J_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}$ measures the instantaneous rate of decrease of the noisy-sample loss. Its change separates into contributions from the output gradients and the features:

$$\frac{d}{dt}\left(\boldsymbol{g}_{\mathbb{L}}^\top J_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}\right) = \underbrace{\dot{\boldsymbol{g}}_{\mathbb{L}}^\top J_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}}_{\text{output-gradient change}} + \underbrace{\boldsymbol{g}_{\mathbb{L}}^\top \dot{J}_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}}_{\text{feature change}} + \underbrace{\boldsymbol{g}_{\mathbb{L}}^\top J_{\mathbb{L}} \dot{\boldsymbol{p}}_{\mathbb{B}}}_{=0 \text{ along a fixed SGD step}}. \tag{9}$$

Dots denote time derivatives along the straight-line SGD step, evaluated at its start. The update direction $\boldsymbol{p}_{\mathbb{B}}$ is fixed during this step, so $\dot{\boldsymbol{p}}_{\mathbb{B}} = 0$.

We isolate the feature contribution and attribute it to the samples providing the update. Writing $\boldsymbol{p}_i = J_i^\top \boldsymbol{g}_i$ and $\boldsymbol{p}_{\mathbb{B}} = \sum_{i \in \mathbb{B}} \boldsymbol{p}_i$, define

$$H_{\text{model},\mathbb{L}} \coloneqq \sum_{j \in \mathbb{L}} \sum_{a=1}^{d_y} (\boldsymbol{g}_j)_a \nabla_{\boldsymbol{\theta}}^2 f_{\boldsymbol{\theta},a}(\boldsymbol{x}_j), \tag{10}$$

$$\dot{J}_{\mathbb{L}}^\top \boldsymbol{g}_{\mathbb{L}} = -H_{\text{model},\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}, \tag{11}$$

$$\boldsymbol{g}_{\mathbb{L}}^\top \dot{J}_{\mathbb{L}} \boldsymbol{p}_{\mathbb{B}} = \sum_{i \in \mathbb{B}} l_i, \qquad l_i \coloneqq -\boldsymbol{p}_i^\top H_{\text{model},\mathbb{L}} \boldsymbol{p}_{\mathbb{B}}. \tag{12}$$

By symmetry of the Hessian, l_i measures how the feature change induced by sample i affects the rate of noise fitting along the batch update. The contributing sample may be clean or noisy; a positive l_i indicates that its feature change facilitates noise fitting. The output gradients retain the original batch-loss normalization.

We accumulate these contributions over training to obtain a score for each sample:

$$C_i(T) = \sum_{\substack{t \text{ before horizon } T \\ i \in \mathbb{B}_t}} \frac{(\Delta t_t)^2}{2} l_{i,t}. \tag{13}$$

Here t indexes SGD updates, i identifies the same dataset sample across batches, and $l_{i,t}$ is evaluated at the start of the update. The factor $(\Delta t_t)^2/2$ weights the contribution according to the second-order expansion of the loss decrease over that step.

Design for disabling feature learning per sample In order to test our mechanism which attributes the FLDD to a feature learning mechanism of a specific group of samples, we need to come up with a specific mechanism that allows allows only subset of samples to do feture learning. Precisely, we seek a design that prevents a subset of training examples $\mathbb{F} \subset \mathbb{S}$ (lets call them frozen samples) not to infleunce the feature kernel of the neural network $f_{\boldsymbol{\theta}}$. Most elegant way is to use Equation 8 to implement a two pathway model where both the $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ term si allowed to change as well as the $\boldsymbol{w}$ in the linear term $(\nabla_{\boldsymbol{\theta}_0} f_{\boldsymbol{\theta}_0})\boldsymbol{w}$. However, such model is unstable and does not converge (more about this and more designs are shown in the Appendix A.3).

We have settled on a two pathway architecture design where the the frozen samples $\mathbb{F}$ can only update the linear readout weights $\boldsymbol{W} \in \mathbb{R}^{(|\boldsymbol{h}(\boldsymbol{x})|+|\boldsymbol{x}|)\times d_y}$ from the concatenation of all the network's hidden activations $\boldsymbol{h}(\boldsymbol{x})$ and the input itself $\boldsymbol{x}$:

$$\bar{f}_{\boldsymbol{\theta},\boldsymbol{W}}(\boldsymbol{x}) = f_{\boldsymbol{\theta}}(\boldsymbol{x}) + (\boldsymbol{h}(\boldsymbol{x}) : \boldsymbol{x})\boldsymbol{W}, \tag{14}$$

where the gradient for parameters $\boldsymbol{\theta}$ is stopped for the frozen samples $\boldsymbol{x} \in \mathbb{F}$. This allows very precisely validate our mechanism.

Testing the mechanism. We do not claim that conditions (i)–(iii) are sufficient for FLDD to arise. Its emergence also depends on the noise fraction, optimizer, batch size, and learning rate. Our aim is to identify the feature-learning contribution to the peak. Algorithm 1 tests this contribution by disabling feature learning for samples with the largest cumulative scores, while preserving the model's ability to fit them through the intervention in Section 3.3. A reduction in the validation-loss peak would support the proposed mechanism.

Algorithm 1 FLDD intervention algorithm

Require: Width sweep $\mathcal{W}$, training horizon T, intervention size n
Ensure: Baseline and intervened validation-loss curves
1: **for** each width $w \in \mathcal{W}$ **do**
2: Train the baseline model to horizon T, accumulating $C_i(T)$ for each sample
3: Select the n samples with the largest $C_i(T)$
4: Retrain from the same initialization, disabling feature learning for the selected samples
5: Keep the data order, training horizon, and all other training settings fixed
6: Record the baseline and intervened validation losses
7: **end for**
8: Compare the two curves across widths, assessing the reduction in the double-descent peak

4 Experiments

References

Alexander Atanasov, Blake Bordelon, and Cengiz Pehlevan. Neural networks as kernel learners: The silent alignment effect, 2021. URL https://arxiv.org/abs/2111.00034.

Francis Bach. High-dimensional analysis of double descent for linear regression with random projections, 2023. URL https://arxiv.org/abs/2303.01372.

Yufei Gu, Xiaoqing Zheng, and Tomaso Aste. Unraveling the enigma of double descent: An in-depth analysis through the lens of learned feature space, 2024. URL https://arxiv.org/abs/2310.13572.

Andi Han, Wei Huang, Zhanpeng Zhou, Gang Niu, Wuyang Chen, Junchi Yan, Akiko Takeda, and Taiji Suzuki. On the role of label noise in the feature learning process. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 21733–21773. PMLR, 13–19 Jul 2025. URL https://proceedings.mlr.press/v267/han25c.html.

Tomoki KUBO and Yusuke IIDA. Analysis of learning process differences for clean and noisy labels in epoch-wise double descent. *Proceedings of the Annual Conference of JSAI*, JSAI2025:3L5GS105–3L5GS105, 2025. doi: 10.11517/pjsai.JSAI2025.0_3L5GS105.

Aitor Lewkowycz and Guy Gur-Ari. On the training dynamics of deep networks with l2 regularization. 2020. URL https://arxiv.org/abs/2006.08643.

Elon Litman and Gabe Guo. A theory of generalization in deep learning, 2026. URL https://arxiv.org/abs/2605.01172.

Matthew Repasky, Xiuyuan Cheng, and Yao Xie. Neural stein critics with staged l^2-regularization, 2023. URL https://arxiv.org/abs/2207.03406.

Rylan Schaeffer, Mikail Khona, Zachary Robertson, Akhilan Boopathy, Kateryna Pistunova, Jason W. Rocks, Ila Rani Fiete, and Oluwasanmi Koyejo. Double descent demystified: Identifying, interpreting & ablating the sources of a deep learning puzzle, 2023. URL https://arxiv.org/abs/2303.14151.

Sidak Pal Singh, Aurelien Lucchi, Thomas Hofmann, and Bernhard Schölkopf. Phenomenology of double descent in finite-width neural networks, 2022. URL https://arxiv.org/abs/2203.07337.

A Appendix

A.1 Identification of the double descent landmarks

We extract the four landmarks from the validation-loss curve at a fixed training horizon, using Algorithm 2. Let ℓ_i denote the final validation loss at width w_i, averaged over seeds when applicable. Smoothing suppresses small fluctuations so that the selected peak reflects the broader shape of the curve.

Algorithm 2 Four-point identification of double descent

Require: Validation losses $\ell_1, \ldots, \ell_m$ at widths $w_1 < \cdots < w_m$, $m \geq 4$
Ensure: Landmarks $\mathcal{D}_T$, if a sufficiently distinct peak is present

```
Smooth ℓ with a Gaussian filter to obtain ℓ̃
Set the contrast threshold δ ← 0.1|ℓ̃_1|
Identify the most prominent peak i_⋆ of ℓ̃ with prominence at least δ
if i_⋆ exists, is dominant, and 3 ≤ i_⋆ ≤ m − 1 then
    Select i_− minimizing ℓ_i over 1 < i < i_⋆
    Select i_+ minimizing ℓ̃_i over i_⋆ < i ≤ m
    if the smoothed peak exceeds both selected minima by at least δ then
        return D_T = (w_1, w_{i_−}, w_{i_⋆}, w_{i_+})
    end if
end if
```

We use Gaussian bandwidth $\sigma = \min\{3, \max\{0.1, m/30\}\}$, measured in grid positions, with constant extension at the boundaries. A peak's prominence is its height above the higher of the two surrounding valley floors, each delimited by a higher peak or the boundary of the curve. A peak is *dominant* if every other peak with prominence at least δ has less than half its prominence. Ties between minima are resolved in favor of the smaller width. The pre-peak minimum uses the observed losses, while the post-peak minimum and both contrast comparisons use the smoothed curve. Curves that do not meet the selection criteria remain unidentified.

This criterion permits local fluctuations between landmarks and does not separately require a decrease from the starting width to the pre-peak minimum. The selected peak describes the validation-loss curve; relating it to an interpolation threshold requires training-loss evidence.

A.2 Derivation of the 2nd order taylor loss changes

$$\mathcal{L}_{\mathbb{Q}}(t) = \mathcal{L}_{\mathbb{Q}}(\theta(t)) \quad \text{Validation loss} \tag{15}$$

$$\theta_t(\tau) = \theta(t) - \tau \nabla_\theta \mathcal{L}_S(\theta(t)) \quad \text{Linear interpolation of one SGD step} \tag{16}$$

$$\mathcal{L}_{\mathbb{Q}}(\theta_t(\Delta t)) = \mathcal{L}_{\mathbb{Q}}(\theta_t(0)) + \int_0^{\Delta t} \dot{\mathcal{L}}_{\mathbb{Q}}(\theta_t(\tau))\, d\tau \quad \text{Approximate with Taylor series} \tag{17}$$

$$\mathcal{L}_{\mathbb{Q}}(\theta_t(\Delta t)) \approx \mathcal{L}_{\mathbb{Q}}(\theta_t(0)) + \Delta t\, \dot{\mathcal{L}}_{\mathbb{Q}}\big|_{\tau=0} + \frac{1}{2}\Delta t^2\, \ddot{\mathcal{L}}_{\mathbb{Q}}\big|_{\tau=0} \quad \text{Approximated at } \tau = 0 \tag{18}$$

$$\dot{\mathcal{L}}_{\mathbb{Q}} = -g_Q^\top J_Q J_S^\top g_S \quad \text{First-order validation change} \tag{19}$$

$$\dot{\mathcal{L}}_{\mathbb{Q}} = -\sum_i \underbrace{\|g_S^{(i)}\|}_{P_i \text{ (Train push)}} \underbrace{g_Q^\top J_Q (J_S^\top)^{(i)} \hat{g}_S^{(i)}}_{T_i \text{ (Transfer)}} \quad \text{Per-sample push and transfer} \tag{20}$$

$$\dot{\mathcal{L}}_{\mathbb{Q}} = \nabla_\theta \mathcal{L}_{\mathbb{Q}}(\theta)^\top \dot{\theta}_t(\tau) \quad \text{Chain rule} \tag{21}$$

$$\ddot{\mathcal{L}}_{\mathbb{Q}} = \nabla_\theta \mathcal{L}_{\mathbb{Q}}(\theta)^\top \ddot{\theta}_t(\tau) + \dot{\theta}_t(\tau)^\top \nabla_\theta^2 \mathcal{L}_{\mathbb{Q}}(\theta) \dot{\theta}_t(\tau) \quad \text{Product rule} \tag{22}$$

$$\text{One gradient step of SGD is linear, thus } \ddot{\theta}_t(\tau) = \vec{0} \tag{23}$$

$$\ddot{\mathcal{L}}_{\mathbb{Q}} = \dot{\theta}_t(\tau)^\top \nabla_\theta^2 \mathcal{L}_{\mathbb{Q}}(\theta) \dot{\theta}_t(\tau) \quad \text{Second-order validation change} \tag{24}$$

$$\nabla_\theta^2 \mathcal{L}_{\mathbb{Q}} = \nabla_\theta \left((\nabla_\theta f)^\top \nabla_f \mathcal{L}_{\mathbb{Q}} \right) \quad \text{Expand the validation Hessian} \tag{25}$$

$$\nabla_\theta^2 \mathcal{L}_{\mathbb{Q}} = (\nabla_\theta f)^\top \nabla_f^2 \mathcal{L}_{\mathbb{Q}} (\nabla_\theta f) + \sum_a (\nabla_f \mathcal{L}_{\mathbb{Q}})_a (\nabla_\theta^2 f)_a \quad \text{Second-order chain rule} \tag{26}$$

$$\begin{aligned} \ddot{\mathcal{L}}_{\mathbb{Q}} = {} & (\nabla_\theta \mathcal{L}_S)^\top \left((\nabla_\theta f)^\top \nabla_f^2 \mathcal{L}_{\mathbb{Q}} (\nabla_\theta f) \right) (\nabla_\theta \mathcal{L}_S) \\ & + (\nabla_\theta \mathcal{L}_S)^\top \left(\sum_a (\nabla_f \mathcal{L}_{\mathbb{Q}})_a (\nabla_\theta^2 f)_a \right) (\nabla_\theta \mathcal{L}_S) \end{aligned} \quad \text{Fixed-feature + feature-learning curvature} \tag{27}$$

$$\underbrace{(\nabla_\theta \mathcal{L}_S)^\top \left((\nabla_\theta f)^\top \nabla_f^2 \mathcal{L}_{\mathbb{Q}} (\nabla_\theta f) \right) (\nabla_\theta \mathcal{L}_S)}_{\text{fixed-feature / loss curvature}}$$

$$+ \underbrace{(\nabla_\theta \mathcal{L}_S)^\top \left(\sum_a (\nabla_f \mathcal{L}_{\mathbb{Q}})_a (\nabla_\theta^2 f)_a \right) (\nabla_\theta \mathcal{L}_S)}_{\substack{\text{feature-learning curvature} \\ \approx 0 \text{ for linear / infinite-width frozen-feature models}}}$$

A.3 Different designs of disabling feature learning on some samples

Training head only

Training a tangent model

A.4 Matrix free formulations

Kernel regression We evaluate the finite-time gradient-descent solution through a spectral filter, approximated using a matrix-free Golub–Kahan reduction of the loss-normalized Jacobian. Jacobian-vector and vector-Jacobian products construct the reduced problem without explicitly forming the Jacobian or kernel matrices. We apply the finite-time filter to the singular values of the reduced problem without ridge regularization.

With respect to the kernel changes: We estimate both quantities using the same fixed Rademacher probes $\boldsymbol{r}_1, \ldots, \boldsymbol{r}_R$:

$$\widehat{D}_t = \sqrt{\frac{\sum_{r=1}^{R} \|(K_t - K_0)\boldsymbol{r}_r\|_2^2}{\sum_{r=1}^{R} \|K_0 \boldsymbol{r}_r\|_2^2}}, \qquad \widehat{S}_t = \sqrt{\frac{\sum_{r=1}^{R} \|(K_t - K_{t^-})\boldsymbol{r}_r\|_2^2}{\sum_{r=1}^{R} \|K_0 \boldsymbol{r}_r\|_2^2}}. \tag{28}$$

A.5 Multistep integrated taylor approximation

A.6 Calculation of cumulative test displacement Δq